

Guia do Projeto: Implementação do Algoritmo Minimax para Pac-Man

Disciplina: Inteligência Artificial

Curso: Ciências da Computação

Professor: Nikson Bernardes Fernandes Ferreira

1. Introdução

Neste projeto desafiador, vocês terão a oportunidade de implementar o **algoritmo Minimax** para controlar o agente Pac-Man em um cenário de jogo dinâmico. O Pac-Man e os fantasmas serão os agentes que interagem no ambiente, e sua tarefa será desenvolver a lógica para que o Pac-Man tome as **melhores decisões possíveis**, antecipando e reagindo aos movimentos dos fantasmas.

2. Objetivo

O principal objetivo deste trabalho é proporcionar uma compreensão aprofundada do algoritmo Minimax. Vocês aprenderão sobre seus **conceitos de minimização e maximização** e como aplicá-los de forma prática para criar um agente inteligente em jogos de soma zero (o ganho de um é a perda de outro).

3. O Algoritmo Minimax

O Minimax é um algoritmo de busca recursivo fundamental em **jogos de dois jogadores e soma zero**, onde os jogadores se revezam em seus turnos. Ele opera sob a premissa de que ambos os jogadores agem de maneira **ideal e estratégica**.

- **Nó Max (Pac-Man):** O jogador maximizador, neste caso o **Pac-Man**, sempre buscará a ação que resulta no estado com a **maior utilidade** (ou seja, a maior pontuação para ele).
- **Nó Min (Fantasmas):** Em contraste, o jogador minimizador, que são os **fantasmas**, escolherá a ação que levará ao estado com a **menor utilidade** (a menor pontuação possível para o Pac-Man).

4. Estrutura do Código

Vocês trabalharão diretamente no arquivo “seuPacManAgents.py”. A lógica do algoritmo Minimax será desenvolvida na função “getAction” da classe “MinimaxAgent”.

A estrutura básica do código já está fornecida para vocês:

```
class MinimaxAgent(MultiAgentSearchAgent):
    def getAction(self, gameState: GameState):
        """Seu código vem aqui
        Adicione o código para minimax
        """

    def minimax(agentIndex=0, depth=0, state=gameState):
        # verifica se jogo acabou, se sim retorna self.evaluationFunction(state) ou
        # betterEvaluationFunction do estado

        # calcula próximo agente
        # calcula próxima profundidade (apenas se agentIndex == self.index a profundidade
        # aumenta)

        # para cada ação possível em state.getLegalActions(agentIndex)
        # calcula o próximo estado com state.generateSuccessor(agentIndex, action)
        # calcula o score chamando minimax recursivamente
        # se for um passo de maximização e o score for maior que o anterior, selecione ele
        # se for um passo de minimização e o score for menor que o anterior, selecione ele
        # retorne a melhor ação
        pass

    return minimax()
```

5. Detalhes da Implementação

A função aninhada minimax é o **coração** da sua implementação. Ela será responsável por gerenciar a recursão, as fases do jogo (turno do Pac-Man vs. turno dos fantasmas) e a tomada de decisões.

5.1. Parâmetros da Função minimax

- **agentIndex:** Indica qual agente está agindo no turno atual.
 - 0 representa o **Pac-Man**.
 - Valores de 1 até numAgents - 1 representam os **fantasmas**.
- **depth:** Representa a **profundidade atual** na árvore de busca Minimax. A profundidade aumenta somente quando todos os agentes completaram suas jogadas.
- **state:** O **estado atual do jogo** (GameState) no qual o algoritmo está avaliando as ações.

5.2. Condição de Parada (Base Case)

A recursão da função minimax deve parar e retornar um valor quando uma das seguintes condições for satisfeita:

- **Jogo Finalizado:** Se o jogo termina (`state.isWin()` ou `state.isLose()`).
- **Profundidade Máxima Atingida:** Se a profundidade atual (`depth`) for igual à profundidade máxima definida (`self.depth`).

Nestes casos, a função deve retornar o valor do estado atual usando a função `self.evaluationFunction(state)`.

5.3. Gerenciamento de Agentes e Profundidade

É crucial que vocês controlem corretamente qual agente está jogando e quando a profundidade da busca deve ser incrementada:

- **Próximo Agente:**
 - Se o `agentIndex` atual for o **último fantasma** (`agentIndex == state.getNumAgents() - 1`), significa que o turno dos fantasmas terminou. O próximo agente a jogar será o **Pac-Man** (0).
 - Caso contrário (se for o Pac-Man ou um fantasma intermediário), o próximo agente será simplesmente `agentIndex + 1`.
- **Próxima Profundidade:**
 - A profundidade da busca (`depth`) **somente deve ser incrementada** quando todos os fantasmas tiverem jogado e o turno do Pac-Man estiver prestes a começar novamente. Isso ocorre especificamente quando o `agentIndex` atual é o último fantasma.

5.4. Lógica de Maximização (Turno do Pac-Man)

Quando o `agentIndex` for 0 (indicando que é o **turno do Pac-Man**), vocês devem implementar a lógica de maximização:

1. Inicialize uma variável para o **valor máximo** com um número negativo muito grande (`-float('inf')`).
2. Inicialize uma variável para armazenar a **melhor ação** encontrada.
3. Itere sobre **todas as ações legais** disponíveis para o Pac-Man no estado atual (`state.getLegalActions(agentIndex)`).
4. Para cada ação, gere o **próximo estado** do jogo utilizando `state.generateSuccessor(agentIndex, action)`.
5. Chame a função minimax **recursivamente** para o próximo estado, passando o **próximo agente** e a **próxima profundidade**. O resultado dessa chamada será o score da ação.
6. Compare o score obtido com o **valor máximo atual**. Se o score for maior, **atualize o valor máximo** e **armazene a ação** correspondente como a melhor ação até então.
7. No **nível mais externo** da função `getAction` (onde a primeira chamada para minimax é

feita), a função deve **retornar a melhor ação** encontrada. Nas chamadas recursivas subsequentes, a função minimax deve **retornar apenas o valor** (o score máximo).

5.5. Lógica de Minimização (Turno dos Fantasmas)

Quando o `agentIndex` for maior que 0 (indicando que é o **turno de um fantasma**), vocês devem implementar a lógica de minimização:

1. Inicialize uma variável para o **valor mínimo** com um número positivo muito grande (`float('inf')`).
2. Itere sobre **todas as ações legais** disponíveis para o fantasma atual no estado (`state.getLegalActions(agentIndex)`).
3. Para cada ação, gere o **próximo estado** do jogo utilizando `state.generateSuccessor(agentIndex, action)`.
4. Chame a função minimax **recursivamente** para o próximo estado, passando o **próximo agente** e a **próxima profundidade**. O resultado dessa chamada será o score da ação.
5. Compare o score obtido com o **valor mínimo atual**. Se o score for menor, **atualize o valor mínimo**.
6. Nas chamadas recursivas, a função minimax deve **retornar o valor mínimo** encontrado. A ação correspondente não precisa ser retornada diretamente, pois o objetivo do fantasma é apenas passar o pior cenário para o Pac-Man.

6. Executando o Programa

Para testar sua implementação do Minimax, abra o terminal na pasta do projeto e utilize o seguinte comando:

```
python3 pacman.py --pacman MinimaxAgent
```

Você também pode ajustar a **profundidade de busca** do algoritmo Minimax, adicionando o parâmetro `--depth`:

```
python3 pacman.py --pacman MinimaxAgent --depth 2
```

Onde `--depth` controla a profundidade da árvore de busca que o algoritmo Minimax explorará. Valores maiores para `--depth` permitirão que o Pac-Man "pense" mais à frente, mas também resultarão em um **tempo de execução significativamente maior**. Comece com profundidades pequenas para depuração!

7. Dicas e Considerações

- **self.evaluationFunction:** A função `self.evaluationFunction(state)` já está implementada e é crucial! Ela é responsável por atribuir uma **pontuação numérica** a um `GameState` específico, servindo como os valores terminais nas folhas da sua árvore Minimax.
- **self.index:** Lembrem-se que `self.index` dentro da classe `MinimaxAgent` sempre se refere ao **índice do Pac-Man**, que é 0.

- **Testes Graduais:** Comecem testando sua implementação com uma **profundidade pequena** (por exemplo, `--depth 1` ou `--depth 2`) para facilitar a depuração e entender o fluxo do algoritmo.
- **Depuração com print:** Usem e abusem de print statements dentro da função minimax para observar o fluxo da recursão, os valores calculados e as decisões tomadas. Isso é vital para identificar erros.
- **Casos Extremos:** Considerem situações onde um agente pode não ter **ações legais** disponíveis (por exemplo, se estiver encurralado). Sua lógica deve ser robusta o suficiente para lidar com esses casos.

8. Entrega

O trabalho deve ser entregue como o arquivo `seuPacManAgents.py` no espaço aluno, contendo a implementação completa e funcional do algoritmo Minimax dentro da classe `MinimaxAgent`.

9. Critérios de Avaliação

A avaliação do seu trabalho será baseada nos seguintes critérios:

- **Correção do Algoritmo:** A implementação do Minimax deve seguir os princípios corretos de maximização e minimização.
- **Desempenho do Pac-Man:** A capacidade do Pac-Man de jogar de forma inteligente e razoável contra os fantasmas, utilizando a estratégia Minimax.
- **Compreensão Conceitual:** A demonstração de que vocês entenderam os conceitos de maximização e minimização no contexto do jogo.
- **Qualidade do Código:** A organização, clareza e comentários no código.